

第 7 章：Policy Gradient (PG, 策略梯度) I：从随机策略到 REINFORCE 与 Monte Carlo Baseline

7.1 本章符号说明

符号/缩写	English	中文	含义
PG	Policy Gradient	策略梯度	直接对策略参数求梯度、最大化期望回报的方法。
REINFORCE	REINFORCE Algorithm	REINFORCE 算法	使用完整轨迹和 Monte Carlo return 估计 Policy Gradient 的基础算法。
MC	Monte Carlo	蒙特卡洛	等待未来奖励被实际采样出来，用样本回报估计期望。
$S_t / A_t / R_{t+l}$	State / Action / Reward Random Variable	状态 / 动作 / 奖励随机变量	时刻 t 的状态、动作，以及执行动作后收到的奖励。小写 s_t, a_t, r_{t+l} 表示一次采样值。
T	Episode Horizon / Terminal Time	轨迹终止时刻	一条 episode 的最后时刻。
$\pi_\theta(a s)$	Parameterized Policy	参数化策略	参数为 θ 的动作条件分布；给定状态 s 后为动作 a 分配概率或概率密度。
θ	Policy Parameters	策略参数	Policy network 中需要学习的参数。
τ	Trajectory	轨迹	一段 state-action-reward 序列。
$p_\theta(\tau)$	Trajectory Probability / Density	轨迹概率 / 密度	当前策略产生整条轨迹 τ 的概率或密度。
$P(s' s,a)$	Transition Probability	状态转移概率	环境从 (s,a) 转移到下一状态 s' 的概率，默认不依赖 θ 。
γ	Discount Factor	折扣因子	$0 \leq \gamma \leq 1$ ，控制较远未来奖励的权重。
G_t	Return / Reward-to-go	从 t 开始的回报	从时刻 t 之后实际采样到的累计折扣奖励。
$J(\theta)$	Policy Objective	策略目标函数	当前策略的期望回报，是策略需要最大化的对象。
$Q_\pi(s,a)$	Action-value Function	动作价值函数	固定 $S_t=s, A_t=a$ 后，对未来随机回报 G_t 取条件期望。
$V_\pi(s)$	State-value Function	状态价值函数	固定 $S_t=s$ 后，按当前策略选择动作时 G_t 的条件期望。
$b_\phi(s)$	Learned Baseline	可学习基线	参数为 ϕ 的状态参考值，用于降低 Policy Gradient 方差。
$A_\pi(s,a)$	Advantage Function	优势函数	$Q_\pi(s,a) - V_\pi(s)$ ，表示动作 a 比状态 s 下当前策略的平均动作好多少。

符号/缩写	English	中文	含义
$\hat{A}_t^{MC}$	Monte Carlo Advantage Estimate	蒙特卡洛优势估计	用单条轨迹的 $G_t - b_\phi(s_t)$ 估计理论 advantage；帽子表示它是估计量。
$f_\theta(s)$	Policy Logits	策略 logits	离散策略网络在 softmax 之前输出的未归一化分数。
$\mu_\theta(s) / \sigma_\theta(s)$	Gaussian Mean / Standard Deviation	高斯均值 / 标准差	连续策略为动作分布输出的参数。
$\mathcal{N}(x; \mu, \sigma)$	Gaussian Probability Density	高斯概率密度	均值为 μ 、标准差为 σ 的高斯分布在 x 处的密度；分号左边是取值，右边是分布参数。
K_t / U_t	Discrete Type / Continuous Parameter Random Variable	离散类型 / 连续参数随机变量	混合动作 $A_t = (K_t U_t)$ 的两个组成部分。
θ_d / θ_c	Discrete-head / Continuous-head Parameters	离散分支 / 连续分支参数	混合策略中 categorical head 和 conditional continuous head 的参数。
$g_{MC}(\tau)$	Monte Carlo Policy-gradient Estimate	蒙特卡洛策略梯度估计	一条采样轨迹 τ 构造出的样本梯度。
L_{sur}	Surrogate Policy Loss	代理策略损失	为了使用自动微分而构造的标量函数；它的负梯度等于希望使用的样本梯度上升方向。
L	Loss Function	损失函数	实现中使用梯度下降最小化的目标。
α_π / α_V	Policy / Baseline Learning Rate	策略 / 基线学习率	分别控制 policy 和 learned baseline 的更新步长。

7.2 本章定位与学习主线

7.2.1 本章解决什么问题

前面的 value-based 方法先学习 $Q(s, a)$ ，再用 $\arg \max_a Q(s, a)$ 选择动作。本章改变建模对象：直接学习动作分布 $\pi_\theta(a|s)$ 。

核心问题是：

动作是从策略分布中采样的，奖励又由不可微的环境产生，如何计算策略参数对期望回报的梯度？

7.2.2 整章推理链

策略是动作条件分布 $\pi_\theta(a|s)$

- > 策略决定轨迹分布 $p_\theta(\tau)$
- > 目标 $J(\theta)$ 是轨迹回报的期望
- > Log-derivative Trick 把“对分布求导”改写成 log-probability 梯度
- > 轨迹分解后只剩 policy 项
- > Reward-to-go G_t 把未来回报分配给当前动作
- > $Q_\pi(s,a)$ 是固定 (s,a) 后 G_t 的条件均值
- > REINFORCE 用采样到的 G_t 估计期望梯度
- > Baseline 在不改变梯度期望的前提下降低方差
- > 选择 $V_\pi(s)$ 作为 baseline 后自然得到 Advantage

7.2.3 本章与第 8 章的边界

本章只使用完整未来奖励构造 Monte Carlo 学习信号：

- REINFORCE 使用 G_t 。
- REINFORCE with baseline 使用 $G_t - b_\phi(s_t)$ 。
- Baseline 可以是可学习网络，但它的监督标签仍是完整 G_t 。

第 8 章才引入 Critic (价值评估器) 和 bootstrap (自举：用当前价值估计构造新的学习目标)，再展开 Temporal Difference (TD, 时序差分)、n-step return (n 步回报) 和 Generalized Advantage Estimation (GAE, 广义优势估计)。

7.3 策略表示：统一看待离散、连续与混合动作

7.3.1 策略是条件分布

随机策略不直接输出“唯一正确动作”，而是输出给定状态下的动作分布：

$$A_t \sim \pi_\theta(\cdot | S_t)$$

训练时需要保存采样动作的 $\log \pi_\theta(A_t | S_t)$ ，因为后面的 Policy Gradient 会用它建立“动作发生的可能性”与“动作之后的回报”之间的联系。

7.3.2 离散动作：Categorical Policy

若动作是有限个互斥选项，例如 left / stay / right，网络先输出 logits：

$$z = f_\theta(s)$$

再用 softmax 得到 categorical distribution (类别分布)：

$$\pi_\theta(a=i|s) = \exp(z_i) / \sum_k \exp(z_k)$$

策略从这个分布采样一个动作。概率之和自动为 1，因此提高一个动作的概率会相对压低其他动作。

7.3.3 连续动作：Diagonal Gaussian Policy

若动作是 d 维连续向量：

$$a = (a_1, a_2, \dots, a_d)$$

常见做法是让网络为每一维输出均值和标准差：

$$\mu_{\theta}(s), \sigma_{\theta}(s) > 0$$

若各维在给定状态后条件独立，联合概率密度分解为：

$$\pi_{\theta}(a|s) = \prod_{j=1}^d \mathcal{N}(a_j; \mu_{\theta_j}(s), \sigma_{\theta_j}(s))$$

两边取对数：

$$\log \pi_{\theta}(a|s) = \sum_{j=1}^d \log \mathcal{N}(a_j; \mu_{\theta_j}(s), \sigma_{\theta_j}(s))$$

符号逐项解释：

- $\mathcal{N}$: 花体大写 N ，表示 Normal Distribution（正态分布，也叫高斯分布）的概率密度函数。
- a_j : 本次实际采样的第 j 维动作值。
- $\mu_{\theta_j}(s)$: 策略网络预测的第 j 维均值。
- $\sigma_{\theta_j}(s)$: 策略网络预测的第 j 维标准差；实现中常输出 `log_std` 再取指数，以保证标准差为正。
- 分号 `;`: 左边是“在哪个动作值处计算密度”，右边是高斯分布的参数。
- $\prod_j$ 变成 $\sum_j \log$: 条件独立时联合密度等于各维密度的乘积，对数把乘积变成求和。

例如二维动作作为“转向、油门”：

$$a = (0.2, 0.7), \mu_{\theta}(s) = (0.1, 0.6), \sigma_{\theta}(s) = (0.2, 0.1)$$

这次动作的 joint log density（联合对数密度）是：

$$\log \pi_{\theta}(a|s) = \log \mathcal{N}(0.2; 0.1, 0.2) + \log \mathcal{N}(0.7; 0.6, 0.1)$$

连续变量取到某个精确数值的概率为 0，因此这里严格来说是 probability density（概率密度），不是离散动作的 probability mass（概率质量）。工程中通常都简称 `log_prob`。

7.3.4 离散加连续：Hybrid Action Policy

混合动作同时包含离散类型和连续参数：

$$A_t = (K_t, U_t)$$

例如：

游戏: K_t = 选择技能, U_t = [瞄准角度, 释放距离]
交易: K_t = {不操作, 买入, 卖出}, U_t = [价格, 数量]

若连续参数的语义由离散类型决定，应使用 conditional factorization（条件分解）：

$$\pi_{\theta}(k, u|s) = \pi_{\theta_d}(k|s) \pi_{\theta_c}(u|s, k)$$

所以联合 log probability 为：

$$\log \pi_{\theta}(k, u|s) = \log \pi_{\theta_d}(k|s) + \log \pi_{\theta_c}(u|s, k)$$

一次采样的顺序是：

state s

- > categorical head 采样离散类型 k
- > 选中 k 对应的 continuous head
- > 从该连续分布采样 u
- > 环境执行联合动作 (k, u)

如果每个离散类型都有自己的连续参数分支，只有本次被选中的分支会通过 $\log \pi_{\theta_c}(u|s, k)$ 获得直接梯度。未选中分支没有对应的实际动作和回报，不应把它们的 log probability 都加进 loss。

7.3.5 有界连续动作

真实动作常有范围约束，例如方向盘角度必须在 $[-1, 1]$ 。直接从无界 Gaussian 采样后用 clip 截断，会改变实际执行分布，使得记录的 log probability 与真实动作不一致。

常见处理是：

1. 从 Gaussian 采样无界变量。
2. 用 tanh 压缩到 $[-1, 1]$ 。
3. 缩放到环境的合法范围。
4. 计算 log probability 时加入 change of variables (变量变换) 带来的 Jacobian correction (雅可比校正)。

本章不推导该校正公式，但要记住原则：动作经过可微变换后，log probability 必须对应变换后的真实分布。

7.4 从策略到轨迹期望目标

7.4.1 轨迹与 Reward-to-go

一条完整轨迹写成：

$$\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T)$$

从时刻 t 开始的 reward-to-go 为：

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}$$

G_t 是随机变量，因为未来状态转移和未来动作都可能随机。

7.4.2 期望回报目标

对从初始状态开始的 episodic task，策略目标可写成：

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [G_0]$$

它表示：按当前策略反复采样轨迹，期望上希望获得更大的 G_0 。

7.4.3 梯度难点在轨迹分布

目标中的轨迹分布 $p_{\theta}(\tau)$ 随策略参数 θ 变化：策略改变动作概率，动作又改变后续状态与奖励。

不能把环境当作普通监督学习网络直接反传，原因包括：

- 离散动作采样不可微。
- 环境转移未必可微，也未必知道其内部函数。
- 奖励可能延迟很多步才出现。

Policy Gradient 的关键不是穿过环境反传，而是对“产生轨迹的概率”求导。

7.5 从轨迹目标到 Policy Gradient

7.5.1 对采样分布求导

对任意可微概率分布，Log-derivative Trick（对数导数技巧，也叫 Score Function Trick）为：

$$\nabla_{\theta} p_{\theta}(x) = p_{\theta}(x) \nabla_{\theta} \log p_{\theta}(x)$$

将它用到轨迹期望：

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \nabla_{\theta} \int p_{\theta}(\tau) G_0 d\tau \\ &= \int \nabla_{\theta} p_{\theta}(\tau) G_0 d\tau \\ &= \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) G_0 d\tau \\ &= \mathbb{E}_{\tau \sim p_{\theta}} [\nabla_{\theta} \log p_{\theta}(\tau) G_0] \end{aligned}$$

这一步把“对随 θ 变化的轨迹分布求导”转成了“在当前分布下采样，再对采样轨迹的 log probability 求导”。

7.5.2 展开轨迹概率

轨迹概率分解为：

$$p_{\theta}(\tau) = p(S_0) \prod_{t=0}^{T-1} \pi_{\theta}(A_t | S_t) P(S_{t+1} | S_t, A_t)$$

取对数后乘积变成求和。初始状态分布 $p(S_0)$ 和环境转移 P 默认不依赖策略参数，因此：

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

代回目标梯度：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau} [\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_0]$$

环境决定哪些轨迹和奖励会被采样，所以环境当然影响梯度样本；“环境项消失”只表示不需要对环境转移函数求导。

7.5.3 用 Reward-to-go 完成 Temporal Credit Assignment（时间归因）

上式让每个时间步的动作梯度都乘上整条轨迹回报 G_0 ，但 A_t 无法影响时刻 t 之前已经发生的奖励。

从期望梯度中去掉这些过去奖励项，不会改变梯度期望，但可以减少噪声。对从 G_0 定义的折扣目标，严格写法为：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau} [\sum_{t=0}^{T-1} \gamma^t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t]$$

外层 γ^t 来自 G_0 中第 t 时刻奖励的折扣位置，而 G_t 内部从当前时刻重新以 γ^0 开始计算。

很多教材和实现在以下情况不显式写外层 γ^t ：

- episodic 任务使用 $\gamma=1$ 。
- 将时间权重吸收进采样分布或目标定义。
- 工程上直接使用 G_t 作为样本权重。

面试时要能说清所采用的目标定义，不要在不同公式约定之间机械比较是否“少了一个 γ^t ”。

7.5.4 对未来随机性取条件期望

即使当前 (S_t, A_t) 完全相同，不同的环境转移和后续动作也会产生不同 G_t 。固定 $S_t=s, A_t=a$ 后对这些未来随机性取平均，定义出：

$$Q_{\pi}(s,a) = \mathbb{E}_{\pi} [G_t | S_t=s, A_t=a]$$

它不是突然换了一个学习信号，而是给“ G_t 的条件均值”起名为 Action-value Function。

对任意一个时间步，根据 iterated expectation（迭代期望，也叫全期望公式）：

$$\begin{aligned} & \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t] \\ &= \mathbb{E}_{S_t, A_t} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \mathbb{E}[G_t | S_t, A_t]] \\ &= \mathbb{E}_{S_t, A_t} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) Q_{\pi}(S_t, A_t)] \end{aligned}$$

已知 (S_t, A_t) 后， $\nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$ 已是确定量，因此可以移出内层条件期望。保留与上一小节一致的时间权重，Policy Gradient 可写成：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}} [\sum_{t=0}^{T-1} \gamma^t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) Q_{\pi}(S_t, A_t)]$$

这一推导给出两个层次：

层次	数学对象	含义
单条轨迹	G_t	本次从 (s_t, a_t) 之后实际观测到的随机回报。
总体期望	$Q_{\pi}(s,a)$	固定 (s,a) 后，所有可能 G_t 的条件平均。

Policy Gradient 的期望表达式中出现 Q_{π} 并不表示必须训练 Q network。下一节会用实际轨迹中的 G_t 估计这个期望梯度。

7.6 REINFORCE：用轨迹样本估计 Policy Gradient

7.6.1 从期望梯度到样本梯度

REINFORCE 不先学习 Q_{π} 。它采样完整轨迹，用本次观测到的 G_t 作为 $Q_{\pi}(s_t, a_t)$ 的 Monte Carlo 样本。

保留严格的折扣时间权重，一条轨迹给出的梯度估计为：

$$g_{MC}(\tau) = \sum_{t=0}^{T-1} \gamma^t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t$$

使用梯度上升更新策略：

$$\theta \leftarrow \theta + \alpha_{\pi} g_{MC}(\tau)$$

到这里只得到了一个希望沿着前进的向量 $g_{MC}(\tau)$ ，还没有定义实现中要最小化的 loss。

7.6.2 从样本梯度构造 Surrogate Loss

自动微分框架的常规接口是：先给定一个标量 loss，计算它的梯度，再用梯度下降最小化它。

因此对已经采样完成的轨迹 τ ，构造 Surrogate Loss（代理损失）：

$$L_{surrogate}(\theta; \tau) = - \sum_{t=0}^{T-1} \gamma^t \log \pi_{\theta}(a_t | s_t) G_t$$

对这个标量求梯度。求导时，轨迹中已采样的 $s_t a_t G_t$ 都当作已知数值，只对 $\log \pi_\theta$ 中的策略参数 θ 求导：

$$\begin{aligned} \nabla_\theta L_{\text{surr}}(\theta; \tau) &= -\sum_{t=0}^{T-1} \gamma^t G_t \nabla_\theta \log \pi_\theta(a_t | s_t) \\ &= -g_{\text{MC}}(\tau) \end{aligned}$$

框架使用梯度下降：

$$\begin{aligned} \theta &\leftarrow \theta - \alpha_\pi \nabla_\theta L_{\text{surr}} \\ &= \theta - \alpha_\pi [-g_{\text{MC}}(\tau)] \\ &= \theta + \alpha_\pi g_{\text{MC}}(\tau) \end{aligned}$$

所以三个对象必须区分：

对象	是什么	关系
$J(\theta)$	真正希望最大化的期望回报	原始优化目标。
$g_{\text{MC}}(\tau)$	一条轨迹给出的 $\nabla_\theta J$ 样本估计	希望沿着它做梯度上升。
$L_{\text{surr}}(\theta; \tau)$	为了调用自动微分而构造的代理损失	$\nabla_\theta L_{\text{surr}} = -g_{\text{MC}}$ 。

L_{surr} 的数值本身不是 $-J(\theta)$ 的 Monte Carlo 估计，也不一定适合跨 batch 比较“越小策略越好”。它是为当前 on-policy 样本构造的局部代理目标，关键是它对 θ 的梯度正确。

看一个单时间步就很直观：

$$L_t = -G_t \log \pi_\theta(a_t | s_t)$$

- $G_t > 0$ ：梯度下降会增大 $\log \pi_\theta(a_t | s_t)$ ，因而提高该动作的概率或密度。
- $G_t < 0$ ：梯度下降会减小 $\log \pi_\theta(a_t | s_t)$ ，因而降低该动作的概率或密度。

G_t 在这个 loss 中是已采样的权重，实现中应当 `detach` 或 `stop_gradient`。对策略依赖采样分布的影响，已经由前面推导的 score-function gradient 处理；不再通过环境或 G_t 反向传播。

7.6.3 完整训练流程

```
initialize policy pi_theta
repeat:
    use current pi_theta to sample complete episodes
    for each episode:
        compute every reward-to-go G_t backward
        read the saved log pi_theta(a_t | s_t)
        accumulate -gamma^t * log pi_theta(a_t | s_t) * G_t
    average the loss over sampled episodes
    update theta with gradient descent
```

一次训练样本的数据流为：


```

s_t
-> policy distribution pi_theta(.|s_t)
-> sample a_t and save log_prob_t
-> environment returns r_{t+1}, s_{t+1}
-> after episode ends, compute G_t
-> loss_t = -gamma^t * log_prob_t * G_t

```

7.6.4 同一个 Loss 如何适配不同动作空间

REINFORCE 的整体 loss 形式不变，变的是 $\log \pi_{\theta}(a_t | s_t)$ 的计算方式：

动作空间	$\log \pi_{\theta}(a_t s_t)$
离散 categorical	被采样类别的 log probability。
多维连续 Gaussian	各维 Gaussian log density 之和。
离散 + 连续混合	离散类型 log probability + 被选中连续分支的 log density。

对混合动作 (k_p, u_p) ：

$$L_t = -\gamma^t G_t [\log \pi_{\theta_k}(k_t | s_t) + \log \pi_{\theta_u}(u_t | s_t, k_t)]$$

同一个 G_t 评价的是整个联合动作，所以它同时为离散决策和已选连续参数提供学习信号。

7.6.5 算法性质

REINFORCE 属于：

- **Policy-based**：直接更新动作分布。
- **On-policy**：训练数据由当前正在更新的策略产生。
- **Monte Carlo**：需要观测完整未来奖励才能得到 G_t 。
- **Model-free**：不需要已知环境的转移函数。
- **No bootstrap**：不用当前价值估计去构造 G_t 。

7.6.6 Monte Carlo 梯度估计器的方差

固定同一 (s_p, a_p) ，后续环境随机性和后续动作采样仍可能产生差异很大的 G_t 。

例如五次从同一 (s, a) 开始得到：

```
G_t = 2, 18, -4, 25, 9
```

样本均值会逐渐接近 $Q_{\pi}(s, a)$ ，但单次更新可能被偶然的 25 或 -4 强烈影响。这就是下一节引入 baseline 的直接动机。

7.7 Baseline：不改变期望梯度的降方差方法

7.7.1 从绝对回报到相对回报

原始 REINFORCE 用 G_t 作为动作的样本权重。但一个回报的绝对数值未必能反映动作相对该状态的平均水平是好还是坏。

例如某个极难状态的平均回报是 -100，某动作这次得到 -60。虽然 -60 仍为负，它其实比该状态的平均表现好 40。

因此将学习信号改为：

$$G_t - b(S_t)$$

baseline 把“绝对回报”变成“相对参考水平的超额回报”。

7.7.2 Action-independent Baseline 不引入梯度偏差

对固定状态 s ，如果 baseline $b(s)$ 不依赖当前动作，则：

$$\begin{aligned} & \mathbb{E}_{A \sim \pi_\theta(\cdot|s)} [\nabla_\theta \log \pi_\theta(A|s) b(s)] \\ &= b(s) \sum_a \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s) \\ &= b(s) \sum_a \nabla_\theta \pi_\theta(a|s) \\ &= b(s) \nabla_\theta \sum_a \pi_\theta(a|s) \\ &= b(s) \nabla_\theta 1 = 0 \end{aligned}$$

连续动作时将求和换成积分，结论不变。因此：

$$\mathbb{E} [\nabla_\theta \log \pi_\theta(A_t|S_t) (G_t - b(S_t))]$$

与不减 baseline 的梯度期望相同。

关键限制是：baseline 在给定 S_t 后不能再依赖当前 A_t 。若随意减去 $b(S_t, A_t)$ ，上述归一化证明不再成立，可能改变期望梯度。

7.7.3 State Value 是自然的 Baseline

对某个状态，当前策略的平均动作价值为：

$$V_\pi(s) = \mathbb{E}_{A \sim \pi_\theta(\cdot|s)} [Q_\pi(s, A)]$$

它正好表示“在该状态下，按当前策略正常行动能得到多少回报”。因此 $V_\pi(s)$ 是判断一个已选动作是否高于当前策略平均水平的自然参考值。

真实 V_π 通常不可知，可以训练 $b_\varphi(s)$ 拟合 Monte Carlo return：

$$L_V(\varphi) = 1/2 \sum_{t=0}^{T-1} [(b_\varphi(S_t) - G_t)^2]$$

因为：

$$\mathbb{E}[G_t | S_t = s] = V_\pi(s)$$

在足够数据和足够表达能力下，回归 G_t 的 baseline 会逼近 $V_\pi(s)$ 。

7.7.4 REINFORCE with Baseline

Monte Carlo advantage estimate 定义为：

$$\hat{A}_t^{MC} = G_t - b_\varphi(S_t)$$

policy loss 改为：

$$L_{policy} = - \sum_{t=0}^{T-1} \gamma^t \log \pi_\theta(a_t | s_t) \hat{A}_t^{MC}$$

baseline loss 为：

$$L_{baseline} = 1 / 2 \sum_{t=0}^{T-1} (b_{\phi}(s_t) - G_t)^2$$

两个优化目标的梯度边界必须清楚：

- 更新 policy 时， $\hat{A}_t^{MC}$ 作为已计算的样本权重，不通过它更新 baseline。
- 更新 baseline 时，用 G_t 做回归标签，不通过 G_t 或 policy 反传。

完整流程：

```
sample complete episodes with current pi_theta
compute G_t for every time step
baseline prediction = b_phi(s_t)
MC advantage estimate = G_t - stop_gradient(b_phi(s_t))
update theta with policy loss
update phi with regression loss (b_phi(s_t) - G_t)^2
```

`stop_gradient` 表示在计算 policy loss 时把 baseline 预测当作已知数值，不让 policy loss 的梯度流入 baseline 参数 ϕ 。

7.7.5 Baseline 的能力边界

- action-independent baseline 保证的是“不改变期望梯度”，不是“任意 baseline 都必然降方差”。
- baseline 需要与 G_t 的状态依赖变化足够相关，否则降方差效果可能很弱。
- baseline 预测很差时，甚至可能增加样本方差。
- baseline 只做中心化，不消除环境随机性和未来动作采样带来的所有噪声。

7.8 Advantage：将动作价值写成“状态水平 + 相对增益”

7.8.1 理论分解

Advantage Function 定义为：

$$A_{\pi}(s,a) = Q_{\pi}(s,a) - V_{\pi}(s)$$

等价地：

$$Q_{\pi}(s,a) = V_{\pi}(s) + A_{\pi}(s,a)$$

这是一个数学分解：

- $V_{\pi}(s)$ 描述状态本身的平均水平。
- $A_{\pi}(s,a)$ 描述特定动作相对这个平均水平好多少。

在当前策略的动作分布下，advantage 的加权平均为 0：

$$\mathbb{E}_{A \sim \pi(\cdot|s)} [A_{\pi}(s,A)] = 0$$

7.8.2 理论 Advantage 与 Monte Carlo 估计

理论量是：

$$A_{\pi}(s,a) = Q_{\pi}(s,a) - V_{\pi}(s)$$

但 REINFORCE with baseline 在一条轨迹中拿到的是：

$$\hat{A}_t^{MC} = G_t - b_{\phi}(s_t)$$

它们不是同一个数学对象：

对象	性质	来源
$A_{\pi}(s,a)$	理论条件期望	真实 $Q_{\pi} - V_{\pi}$ 。
$\hat{A}_t^{MC}$	带噪声的单样本估计	一次 G_t 减去估计 baseline。

若 b_{ϕ} 很好地近似 V_{π} ，则在固定 (s,a) 下反复采样：

$$\mathbb{E} [\hat{A}_t^{MC} | S_t=s, A_t=a] \approx A_{\pi}(s,a)$$

7.8.3 与 Actor-Critic 的分界

REINFORCE with baseline 和 Actor-Critic 的 policy loss 都可能写成：

$$L_{policy} = -\log \pi_{\theta}(a_t | s_t) \hat{A}_t$$

分界不在 loss 外观，而在 $\hat{A}_t$ 的构造方式：

方法	学习信号	Bootstrap	更新时机
REINFORCE	G_t	否	通常等完整 episode。
REINFORCE with baseline	$G_t - b_{\phi}(s_t)$	否	通常等完整 episode。
Actor-Critic	TD error、n-step 或 GAE	通常是	可以在较短 rollout 后更新。

有些资料会将任意“带可学习 value baseline 的策略梯度”宽泛地称为 Actor-Critic。为了让学习主线清晰，本课程使用以下边界：

- 只用完整 G_t 训练 baseline：仍归入 REINFORCE with baseline。
- 价值估计器开始使用 Bellman bootstrap、TD 或 n-step target：进入第 8 章 Actor-Critic。

7.9 端到端例子

7.9.1 离散动作：Softmax Policy 更新

某状态下两个动作的当前概率为：

动作	当前概率
left	0.4
right	0.6

本次采样到 `left`，且 $G_t=10$ 。

- 无 baseline 时，最小化 $-\log\pi(\text{left}|s) \times 10$ 会提高 **left** 的概率。
- 若 baseline 为 7，则 $\hat{A}_t^{MC}=3$ ，仍提高 **left** 的概率，但更新强度取决于它高出参考水平多少。
- 若 baseline 为 12，则 $\hat{A}_t^{MC}=-2$ ，说明它虽然获得正回报，却低于该状态的预期水平，因此会降低 **left** 的概率。

7.9.2 连续动作：Gaussian Policy 更新

假设单维动作是方向盘角度，策略输出：

$$\mu_{\theta}(s)=0.1, \sigma_{\theta}(s)=0.2$$

本次采样 $a_t=0.3$ ，并得到正的 $\hat{A}_t^{MC}$ 。最小化：

$$-\log\mathcal{N}(0.3;0.1,0.2) \hat{A}_t^{MC}$$

会提高在该状态下采样到 0.3 附近动作的密度。梯度可能同时调整均值和标准差，不是简单把均值直接改成 0.3。

7.9.3 混合动作：技能与释放参数

假设策略先采样：

```
k_t = 火球术
u_t = [角度 20°, 距离 8]
```

训练信号为：

$$-\hat{A}_t^{MC} [\log\pi_{\theta_a}(\text{火球术}|s_t) + \log\pi_{\theta_c}([20,8]|s_t, \text{火球术})]$$

若 $\hat{A}_t^{MC}>0$ ：

- 离散分支会提高相似状态下选择火球术的概率。
- 火球术对应的连续分支会提高采样类似角度和距离的密度。
- 其他未选中技能的连续参数分支不获得这次动作的直接梯度。

7.9.4 理论 Advantage 与单次估计

某状态有三个动作：

动作	$\pi(a s)$	$Q_{\pi}(s,a)$
left	0.5	12
stay	0.3	8
right	0.2	4

状态价值为：

$$V_{\pi}(s)=0.5 \times 12+0.3 \times 8+0.2 \times 4=9.2$$

理论 advantage 为：

$$\begin{aligned} A_{\pi}(s, \text{left}) &= 12-9.2=2.8 \\ A_{\pi}(s, \text{stay}) &= 8-9.2=-1.2 \\ A_{\pi}(s, \text{right}) &= 4-9.2=-5.2 \end{aligned}$$

若本次采样 `left`，实际 $G_t=15$ ，baseline 预测 $b_\phi(s)=9$ ，则：

$$\hat{A}_t^{MC} = 15 - 9 = 6$$

这个 6 不等于理论 $A_\pi(s, left)=2.8$ ，而是单条轨迹给出的带噪声估计。

7.10 章末统一视图

7.10.1 数学对象的层次

符号	层次	是样本还是期望	用途
G_t	单条轨迹	样本	REINFORCE 直接使用的 Monte Carlo return。
$Q_\pi(s, a)$	固定状态动作	条件期望	表示特定动作的平均未来回报。
$V_\pi(s)$	固定状态	条件期望	表示当前策略在该状态的平均水平。
$A_\pi(s, a)$	固定状态动作	条件期望	表示动作相对状态平均水平的增益。
$\hat{A}_t^{MC}$	单条轨迹	样本估计	实现中用于 policy loss 的带噪声优势估计。

7.10.2 从建模到更新的完整闭环

1. policy network 输出动作分布
2. 从分布采样动作并保存 `log_prob`
3. 环境生成奖励和下一状态
4. episode 结束后反向计算 `G_t`
5. 可选: baseline 从状态预测参考回报
6. 构造权重 `G_t` 或 `G_t - b_phi(s_t)`
7. 用 `-log_prob * weight` 更新 policy
8. 若有 baseline, 用 `(b_phi(s_t) - G_t)^2` 单独更新它

7.11 本章面试高频问题

7.11.1 Policy Gradient 为什么使用 $\nabla \log \pi$?

Log-derivative Trick 将 $\nabla p_\theta(\tau)$ 改写为 $p_\theta(\tau) \nabla \log p_\theta(\tau)$ ，使梯度可以写成当前轨迹分布下的样本期望。展开轨迹 log probability 后，只剩各时刻的 $\nabla \log \pi_\theta(A_t | S_t)$ 。

7.11.2 G_t 和 $Q_\pi(s, a)$ 是什么关系?

G_t 是一条具体轨迹上的随机回报； $Q_\pi(s, a) = \mathbb{E}[G_t | S_t=s, A_t=a]$ 是固定 (s, a) 后对所有可能未来取平均。REINFORCE 用实际采样的 G_t 作为 $Q_\pi(s, a)$ 的 Monte Carlo 样本。

7.11.3 Reward-to-go 在推导中的作用是什么?

它将当前动作无法影响的过去奖励从当前梯度权重中去掉，保留当前动作之后的奖励。这是基于因果关系的 credit assignment，可以在不改变梯度期望的前提下降低方差。

7.11.4 Baseline 为什么不改变期望梯度?

给定状态后，若 baseline 不依赖当前动作，则 $\mathbb{E}_{A \sim \pi}[\nabla \log \pi(A|s)b(s)] = 0$ 。因此减去 baseline 只会改变样本梯度的中心和方差，不改变期望。

7.11.5 为什么常用 $V_\pi(s)$ 作为 Baseline?

$V_\pi(s)$ 是在状态 s 下按当前策略选择动作的平均未来回报，正好是判断已选动作相对当前策略平均水平好坏的参考点。它未知时，可用 $b_\phi(s)$ 回归 Monte Carlo return G_t 。

7.11.6 理论 Advantage 和 $G_t - b_\phi(s_t)$ 是同一个东西吗?

不是。 $A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s)$ 是理论条件期望； $G_t - b_\phi(s_t)$ 是一条轨迹上的 Monte Carlo 估计。当 baseline 近似 V_π 时，后者的条件期望会近似前者。

7.11.7 REINFORCE with Baseline 与 Actor-Critic 的边界是什么?

本课程按学习信号区分：如果 baseline 只回归完整 G_t ，不 bootstrap，仍属于 REINFORCE with baseline；当价值估计器用 TD、n-step 或其他 bootstrap target 构造 advantage 时，进入 Actor-Critic。

7.12 本章练习

1. 从 $J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[G_0]$ 推导到 trajectory-level Policy Gradient。
2. 展开 $p_\theta(\tau)$ ，说明环境转移项为什么不需要求导，以及环境为什么仍然影响梯度样本。
3. 说明从 G_0 换成 reward-to-go G_t 的因果逻辑。
4. 使用 iterated expectation 说明 $Q_\pi(s, a)$ 如何从 G_t 的条件期望中出现。
5. 证明 action-independent baseline 不改变 Policy Gradient 的期望。
6. 说明为什么 $V_\pi(s)$ 是自然 baseline，以及真实 V_π 未知时如何训练近似。
7. 对比 $A_\pi(s, a)$ 与 $\hat{A}_t^{MC}$ 。
8. 写出二维 diagonal Gaussian policy 的 joint log probability。
9. 对混合动作 $A_t = (K_p U_p)$ ，写出 conditional policy factorization 和对应 REINFORCE loss。
10. 说明 REINFORCE with baseline 与 Actor-Critic 的分界标准。

▼ 参考答案 (点击展开)

1. **Trajectory-level 推导**：先将期望写成积分，得到 $\nabla J = \int \nabla p_\theta(\tau) G_0 d\tau$ ；使用 $\nabla p = p \nabla \log p$ 得到 $\mathbb{E}[\nabla \log p_\theta(\tau) G_0]$ 。
2. **轨迹分解**： $p_\theta(\tau) = p(S_0) \prod_t \pi_\theta(A_t|S_t) P(S_{t+1}|S_t, A_t)$ 。 $p(S_0)$ 和 P 不依赖 θ ，所以其 log-gradient 为 0；但环境转移仍决定哪些轨迹、状态和奖励会被采样，因此影响梯度样本的分布。
3. **Reward-to-go**：时刻 t 的动作无法影响过去奖励，将过去奖励乘到当前 score function 上只会带来噪声。保留动作之后的 G_t 能进行时间 credit assignment。
4. **Q_π 的来源**：固定 (S_p, A_p) 后，policy score 已是确定量。根据迭代期望， $\mathbb{E}[\nabla \log \pi G_t] = \mathbb{E}[\nabla \log \pi \mathbb{E}[G_t|S_p, A_p]]$ ，而内层期望定义为 $Q_\pi(S_p, A_p)$ 。
5. **Baseline 证明**：给定 s ， $b(s) \sum_a \pi_\theta(a|s) \nabla \log \pi_\theta(a|s) = b(s) \nabla \sum_a \pi_\theta(a|s) = b(s) \nabla 1 = 0$ 。
6. **V_π Baseline**： $V_\pi(s) = \mathbb{E}_{A \sim \pi}[Q_\pi(s, A)]$ 是该状态下当前策略的平均回报水平。未知时可训练 $b_\phi(s)$ 最小化 $(b_\phi(s_t) - G_t)^2$ 。
7. **Advantage 对比**： $A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s)$ 是理论期望量； $\hat{A}_t^{MC} = G_t - b_\phi(s_t)$ 是单条轨迹上的带噪声估计。
8. **二维 Gaussian**： $\log \pi_\theta(a|s) = \log \mathcal{N}(a_1; \mu_{\theta,1}(s), \sigma_{\theta,1}(s)) + \log \mathcal{N}(a_2; \mu_{\theta,2}(s), \sigma_{\theta,2}(s))$ 。
9. **混合动作**： $\pi_\theta(k, u|s) = \pi_\theta(k|s) \pi_\theta(u|s, k)$ ；样本 loss 为 $-\gamma^t G_t \log \pi_\theta(k_t|s_t) + \log \pi_\theta(u_t|s_t, k_t)$ 。
10. **算法边界**：只用完整 Monte Carlo return G_t 构造 policy 和 baseline 信号时，归入 REINFORCE with baseline；当价值估计器用 TD、n-step 或 Bellman bootstrap target 构造 advantage 时，进入 Actor-Critic。